Customer Churn Prediction Using Machine Learning

AI for Engineers – Project Report

Name: Abhinav Kumar
Roll No: 1024031186

Group: 2C81

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

1. Introduction

Customer churn is one of the most critical challenges faced by subscription-based digital and telecom services.
Churn refers to customers discontinuing their service subscription within a given time period.
High churn rates significantly affect revenue streams, customer lifetime value, brand loyalty, and long-term profitability.
In competitive industries, retaining existing customers is more cost-effective than acquiring new ones.
Therefore, businesses increasingly rely on predictive analytics to identify customers who are at risk of leaving.

Machine learning provides a scalable and data-driven solution to analyze historical customer data and predict future churn behavior.
This project focuses on building a classification-based machine learning model to forecast churn probability and support proactive business strategies.

2. Problem Statement

Organizations struggle to manually identify customers who are likely to churn.
Traditional statistical methods often fail to capture complex relationships between demographic factors, billing information, and service usage behavior.
IBM Watson Analytics proposes a machine learning approach to analyze telecom customer data and predict churn risk.
The goal is to transform raw customer data into actionable insights that enable companies to reduce attrition and improve retention performance.

3. Objectives

The primary objectives of this project are:

• To analyze telecom customer behavioral patterns.
• To identify key factors influencing churn decisions.
• To develop supervised classification models.
• To evaluate model performance using multiple evaluation metrics.
• To interpret results and derive business insights.
• To propose data-driven retention strategies based on predictive modeling.

4. Dataset Description and Source

The dataset used in this project is the Telco Customer Churn Dataset obtained from Kaggle.

Dataset Characteristics:
• Total Records: 7043 customers
• Features: More than 20 attributes
• Target Variable: Churn (0 = No, 1 = Yes)

The dataset contains customer demographic details, service subscription information, billing data, contract type, and payment method.
This real-world dataset is suitable for supervised classification modeling and churn prediction tasks.

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.30	1840.75	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	70.70	151.65	Yes

5 rows × 21 columns

Figure 0: Dataset Preview (First 5 Rows)

5. Data Preprocessing

Data preprocessing is a crucial step in machine learning model development.
The following steps were performed:

1. Removed the irrelevant feature 'customerID'.
2. Converted the 'TotalCharges' column to numeric format.
3. Handled missing values by removing incomplete records.
4. Encoded categorical variables using Label Encoding.
5. Applied feature scaling using StandardScaler.
6. Split the dataset into 80% training and 20% testing sets.

Standardization Formula Used:

$$Z = (X - \mu) / \sigma$$

Where:
μ = Mean of the feature
σ = Standard deviation

Preprocessing ensures clean, normalized, and model-ready data.

6. Methodology and Flow Process

The project followed a structured machine learning pipeline:

Data Collection → Data Cleaning → Feature Encoding → Feature Scaling →
Train-Test Split → Model Training → Model Evaluation → Result Interpretation.

This systematic approach ensures reproducibility, transparency, and efficient experimentation.
Each stage contributes to improving the reliability and interpretability of the predictive model.

7. Model Development

Two supervised classification algorithms were implemented:

1. Logistic Regression:
A baseline linear classification model used for binary prediction.
Probability function:
P(Y=1) = 1 / (1 + e^(-z))

2. Random Forest Classifier:
An ensemble learning technique that builds multiple decision trees and aggregates their predictions using majority voting.
Random Forest handles non-linear relationships and reduces overfitting compared to single decision trees.

8. ROC Curve Analysis

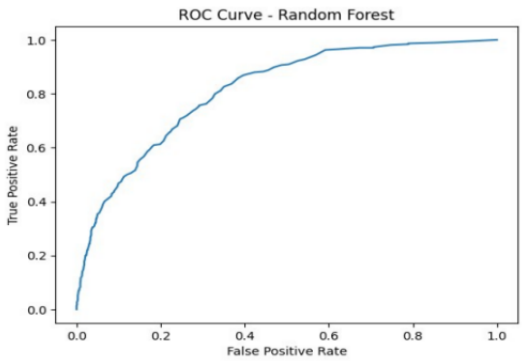


Figure 4: ROC Curve – Random Forest

8. Performance Evaluation Metrics

Model performance was evaluated using multiple metrics:

Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1-Score = 2 × (Precision × Recall) / (Precision + Recall)
ROC-AUC for overall classification separability.

Using multiple metrics ensures balanced evaluation beyond simple accuracy, particularly in imbalanced datasets.

5. Logistic Regression Results

Logistic Regression Accuracy: 0.7853589196872779

	precision	recall	f1-score	support
0	0.83	0.89	0.86	1033
1	0.62	0.49	0.55	374
accuracy			0.79	1407
macro avg	0.73	0.69	0.70	1407
weighted avg	0.77	0.79	0.78	1407

Figure 1: Logistic Regression Classification Report (Accuracy: 78.5%)

9. Experimental Results

Logistic Regression Accuracy: 78.5%
Random Forest Accuracy: 78.2%

Observations:
• High predictive performance for non-churn customers.
• Lower recall (~49%) for churn class due to class imbalance.
• Both models show comparable performance and stability.
• ROC curve indicates strong model separability and satisfactory AUC.

6. Random Forest Results

Random Forest Accuracy: 0.7825159914712153

	precision	recall	f1-score	support
0	0.82	0.89	0.86	1033
1	0.62	0.47	0.54	374
accuracy			0.78	1407
macro avg	0.72	0.68	0.70	1407
weighted avg	0.77	0.78	0.77	1407

Figure 2: Random Forest Classification Report (Accuracy: 78.2%)

10. Interpretation and Business Insights

The confusion matrix reveals high true negative rates and moderate false negatives.
This indicates that while the model performs well for the majority class, improvement is needed in detecting churn cases.

Business Implications:

- Identify high-risk customers before churn occurs.
- Offer personalized retention incentives.
- Improve service quality for vulnerable segments.
- Reduce long-term revenue loss through proactive intervention.

7. Confusion Matrix Analysis

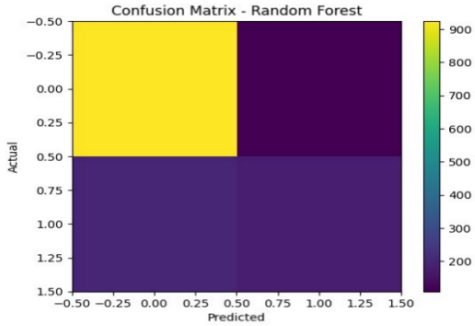


Figure 3: Confusion Matrix – Random Forest

11. Limitations

- Class imbalance impacts recall for churn class.
- Limited hyperparameter tuning was performed.
- Model performance may vary across industries.
- Advanced ensemble methods could further enhance prediction accuracy.

12. Future Scope

- Apply SMOTE technique for class balancing.
- Perform hyperparameter tuning using GridSearchCV.
- Implement advanced algorithms such as XGBoost.
- Deploy the model as a web-based predictive dashboard.
- Integrate real-time churn monitoring systems.

13. Conclusion

This project successfully developed and evaluated classification-based machine learning models to predict customer churn.
The models achieved approximately 78% accuracy on unseen test data.
The findings demonstrate that predictive analytics can effectively transform customer data into actionable insights.

Machine learning offers scalable, data-driven solutions for reducing churn, improving retention strategies, and supporting strategic business decision-making in subscription-based services.

Implementation Code

DATA FILE

```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving Telco-Customer-Churn.csv to Telco-Customer-Churn.csv
```

REQUIRED LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, roc_curve, roc_auc_score
```

LOAD DATASET

```
df = pd.read_csv('Telco-Customer-Churn.csv')

print(df.shape)
df.head()
```

```
(7043, 21)
```

{"type":"dataframe","variable_name":"df"}

DATA CLEANING

```
# Convert TotalCharges to numeric
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'],
errors='coerce')

# Drop missing values
df = df.dropna()
```

ENCODE CATEGORICAL COLUMNS

```
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

DEFINE FEATURES & TARGET

```
X = df.drop('Churn', axis=1)
y = df['Churn']
```

TRAIN-TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

FEATURE SCALING

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

LOGICAL REGRESSION

```
lr = LogisticRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test,
y_pred_lr))
print(classification_report(y_test, y_pred_lr))

Logistic Regression Accuracy: 0.7853589196872779
              precision    recall  f1-score   support

           0       0.83      0.89      0.86      1033
           1       0.62      0.49      0.55       374

    accuracy                           0.79      1407
   macro avg       0.73      0.69      0.70      1407
weighted avg       0.77      0.79      0.78      1407
```

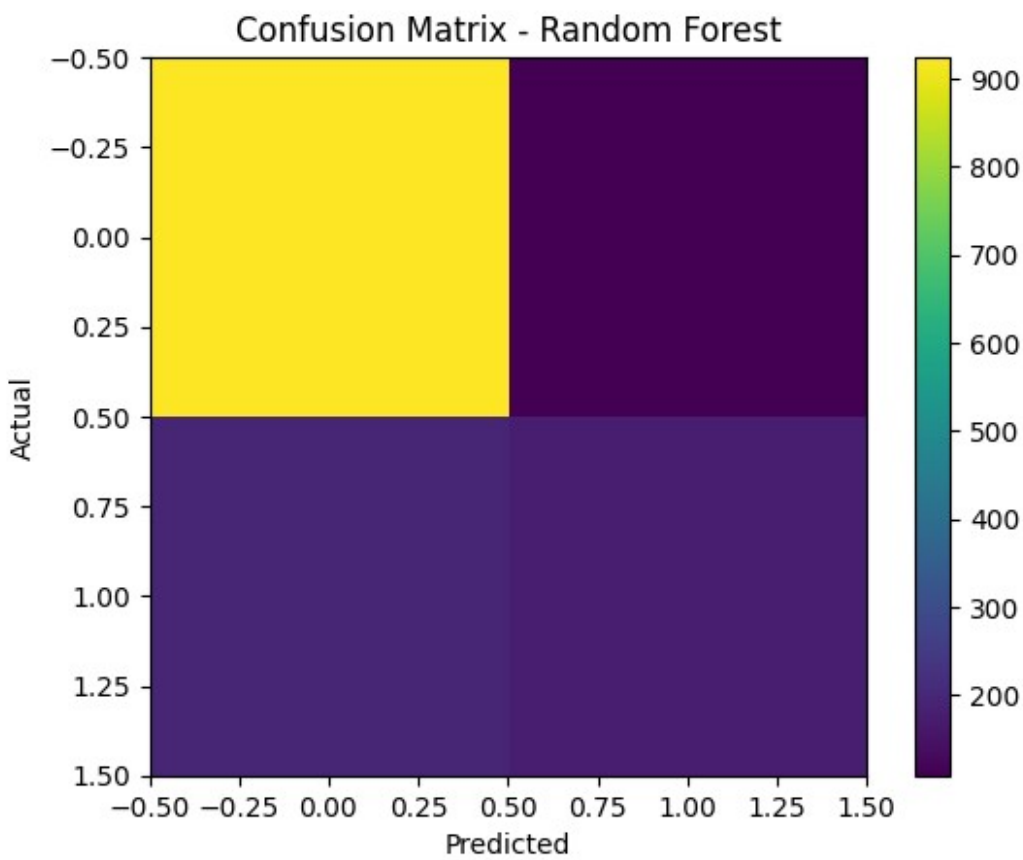
RANDOM FOREST

```
rf = RandomForestClassifier()
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.7825159914712153
              precision    recall  f1-score   support

           0       0.82      0.89      0.86      1033
           1       0.62      0.47      0.54       374

    accuracy                           0.78      1407
   macro avg       0.72      0.68      0.70      1407
weighted avg       0.77      0.78      0.77      1407
```

CONFUSION MATRIX

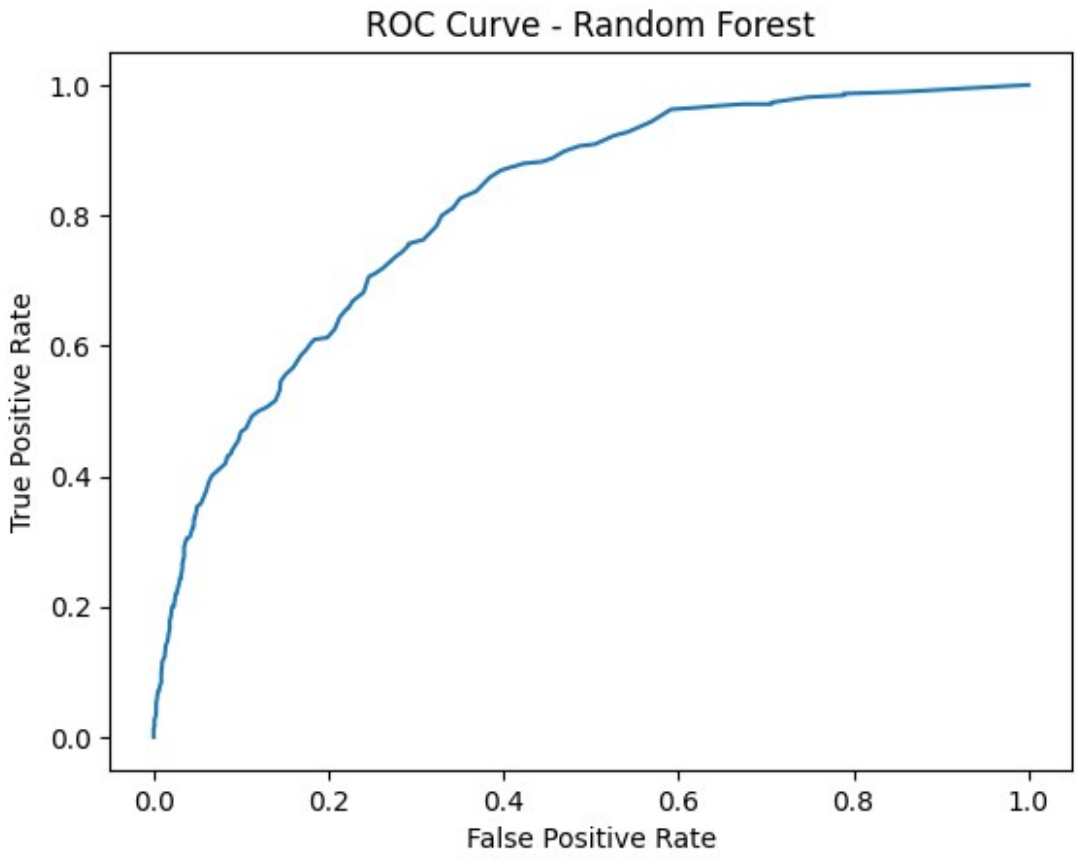
```
cm = confusion_matrix(y_test, y_pred_rf)

plt.figure()
plt.imshow(cm)
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()
```



ROC CURVE

```
y_prob_rf = rf.predict_proba(X_test)[:,1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob_rf)
auc_score = roc_auc_score(y_test, y_prob_rf)

print("AUC Score:", auc_score)

plt.figure()
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.show()
```

```
AUC Score: 0.8128691677322166
```



END